

Survival Model Evaluation: flchain

Fitted with the survival-analysis-pipeline.

Source data	<code>datasets/flchain.csv.gz</code>
Rows	7,871 (27.5% with the ending observed, 72.5% censored)
Start dates	1995-01-01 to 2003-01-01
Evaluation	3 expanding-window temporal folds
Source of figures	<code>reports/flchain_demo/metrics.json</code> , regenerated by the command in section 9

1. Summary

This report evaluates two survival models fitted to `flchain.csv.gz`. It holds 7,871 rows, each observed from its start date. 27.5% have observed endings and 72.5% are censored, still running when observation stopped. Median observed duration, censored rows included, is 4303 days. The models predict, from what was on file at the start date, how long each row survives.

The like-for-like comparison is the fold mean. Each of the 3 temporal folds trains both models on one stretch of history and tests them on the next, and the 3 scores average. On concordance, the share of pairs a ranking orders correctly where 0.500 is a coin flip, XGBoost AFT scores 0.769 and the Cox proportional hazards baseline 0.795. The Cox baseline scores higher.

Pooled over 4,723 out-of-time test rows, the AFT model scores 0.743 (95% bootstrap interval 0.728 to 0.757). Section 5 explains how the two figures differ.

Both models are saved in one bundle, which records the Cox baseline as recommended for scoring new rows.

The pooled figure splits between `flc_band` group membership and ranking within it. The check behind that reading replaces each row's prediction with the average prediction of its group, so every row in a group ties and only the order of groups is scored. That ordering alone scores 0.653. Restricted to pairs of rows inside the same group, the pairs a group average leaves tied, the boosted model scores 0.689, clear of the coin flip. Section 5 gives the decomposition.

The boosted model's probabilities lose to a no-skill forecast at 365, 730, and 1460 days. A no-skill forecast assigns every row the same population-average probability. The limitations section says what remains usable.

This dataset has no known generating process, so there is no known best-achievable score to compare these against, and feature attributions cannot be checked against a true mechanism.

2. Motivation

From the run's notes, written by the analyst.

The serum free light chain cohort is one of the most widely taught survival datasets. It ships with R's survival package, it appears in the standard Python tutorials, and a reader who has studied survival analysis likely arrives already knowing roughly what a model scores on it. That makes it a useful public test. The pipeline's numbers can be checked against reported ones, and where they differ, the difference has to be explainable.

The data is also a hard test. Nearly three quarters of the subjects were still alive when the study stopped watching, so a model here must learn to predict lifetimes from lifetimes it mostly never saw end. How each of the two models holds up under that much censoring is the run's main finding, and the interpretation section reports it.

3. Data

Durations are measured in days. Columns dropped before fitting: `flc_band`.

Left truncation, where a row was already running when the source's records begin, is a data fault. Its recorded start is not its true start and nothing marks it. This pipeline has no delayed-entry handling, so those rows must be excluded during preparation. Whether they were excluded is recorded in the dataset's own documentation.

From the run's notes, written by the analyst.

Each row is one subject in the Mayo Clinic serum free light chain study, observed from their blood sample until death or the end of follow-up. The features are the intake measurements: age at sampling, sex, the kappa and lambda free light chain concentrations, serum creatinine, and the assay decile group, the study's own one-to-ten grouping of the light-chain result. A flag for a prior diagnosis of MGUS, a benign precursor blood condition (monoclonal gammopathy of undetermined significance), completes the set. Only 28% of subjects died during follow-up. For everyone else, the data records how long they had lived so far, and nothing more.

The recorded start date is a year, with no month or day. Two subjects sampled in the same year cannot be put in time order. The preparation script therefore shuffles within each year with a fixed seed. That makes the

order genuinely arbitrary instead of the source file's hidden age sort. The limitations section carries the consequences for folds and labels.

The measured clock starts at the blood sample for every subject by construction, so no subject is first seen partway through the interval being measured, and the left-truncation fault described above does not arise in this dataset.

Figure 1 shows Kaplan-Meier survival curves by `flc_band`, the coarsest structure in the outcome before any model is fitted.

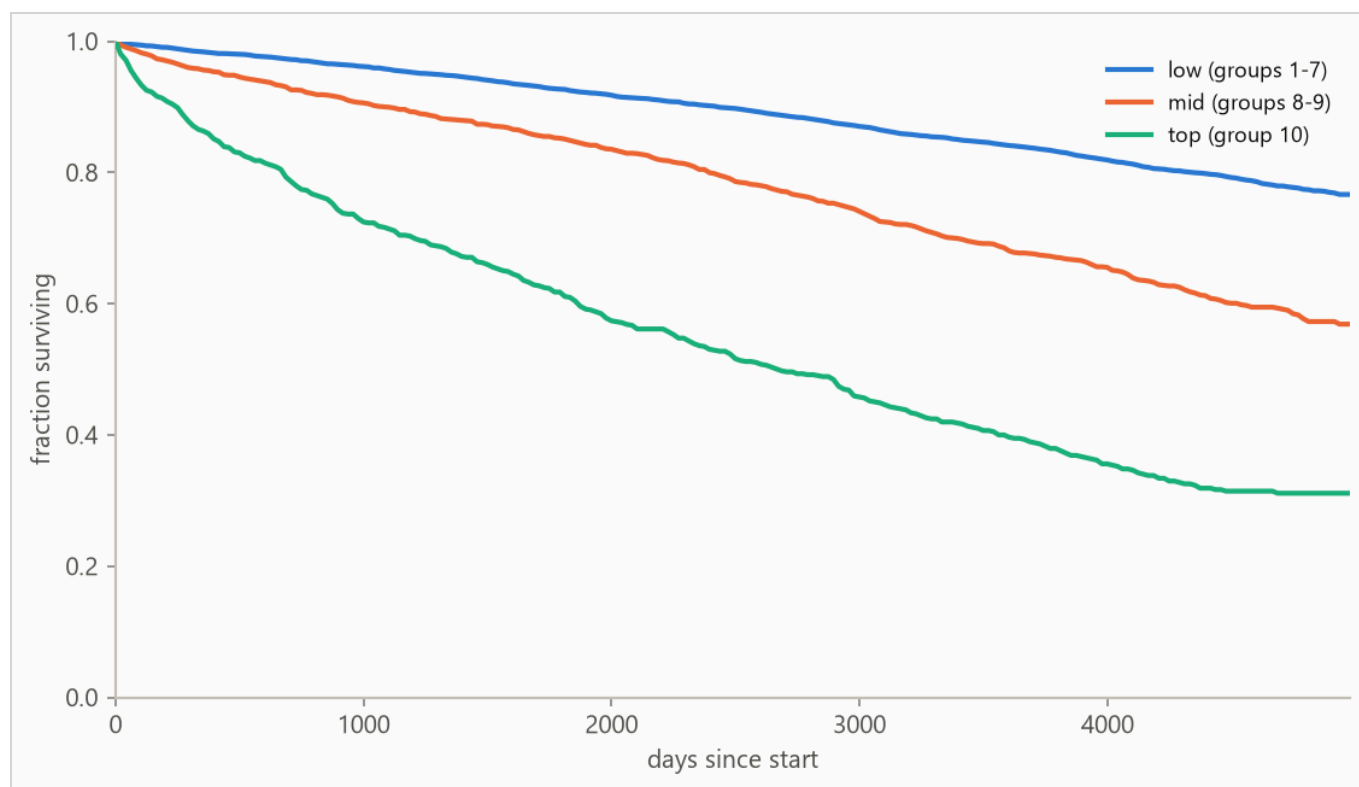


Figure 1. Kaplan-Meier survival curves by `flc_band`: the fraction of each group still running at each age, with censored rows counted for as long as they were observed. Groups beyond the seven most frequent, where present, are collapsed into (other) for this plot only.

4. Method

4.1 Model class

The pipeline fits two models on every run. The first is a boosted-tree accelerated failure time (AFT) model, XGBoost with its `survival:aft` objective, built for durations with incomplete observations. An observed ending tells the model the exact lifetime, and a censored row tells it only "at least this long", so every row

contributes. It predicts each row's median survival time in days, and a log-normal curve around that median, whose width is fitted once and shared by every row, gives the probability of surviving any horizon. The second is a Cox proportional hazards baseline, the standard linear survival model, fitted with lifelines' `CoxPHFitter` on the same features to answer whether the boosted model was necessary. The run recommends whichever scores the higher fold-mean concordance.

The two differ in what they assume. The AFT model assumes every row follows the same survival curve run on a faster or slower clock, so features change when things happen, never the curve's shape. The Cox model makes no assumption about that shape at all, reading the curve from the data by counting who was still running at each age. In exchange it assumes each feature multiplies risk by the same factor at every age, which this report does not test. Which set of assumptions suits a dataset cannot be known in advance, which is why both are fitted and scored.

Numeric features pass through, missing values included (the Cox baseline gets train-window median imputation). Text columns are one-hot encoded with the vocabulary refit per training window, so a fold's features reflect only what was on file by its split date.

4.2 Temporal validation and label re-censoring

Evaluation uses 3 expanding-window folds ordered by start date: the earliest 40% of rows is burn-in, trained on, never tested, and each fold trains on every row started before its split date and tests on the next block (sizes in Table 2).

Training labels are re-censored at each split date. A row started long before a split may have died after it, and its label contains that future, so every post-split death is rewritten as a censoring at the split. Omitting this raises scores by importing the future, which makes it the most consequential detail in the pipeline. It is a standalone function (`temporal_folds.recensor`) with dedicated tests.

4.3 Selection and calibration

The boosted model's hyperparameters are selected once on the first fold's training window by an inner temporal split, the same past-then-future cut made inside that window. Concordance grades only whether rows are ordered correctly, and a model can order them well while drawing survival curves far too wide or too narrow. So selection is scored on held-out censored log-likelihood instead, which grades the whole predicted distribution against what was observed.

The predictive scale is the width of the boosted model's log-normal curve, one unitless number shared by every row. A larger scale spreads the model's probability over a wider range of lifetimes. Training used 1.2, a setting of the loss that shapes how the medians are fitted. Then, with the medians held fixed, the width that best matches held-out outcomes on the training window's most recent stretch was measured at 1.46 and

carried to the model refitted on the full window. The two answer different questions, so they need not agree. Every probability the boosted model reports here uses the measured value.

The methodology is validated separately against synthetic data with known ground truth.

5. Results

5.1 Discrimination

Table 1. Concordance index by model, pooled over 4,723 test rows with 95% percentile bootstrap intervals over 500 resamples. Higher is better, and 0.500 is a coin flip. Fold-mean rows score each of the 3 folds separately and average. The interval resamples test rows with the fitted models held fixed, so it measures scoring precision, not stability across history. The fold spread is the guide to that.

METHOD	CONCORDANCE (HARRELL'S C)	95% INTERVAL
XGBoost AFT (pooled)	0.743	0.728 to 0.757
XGBoost AFT (fold mean)	0.769	not resampled
Cox proportional hazards (fold mean)	0.795	not resampled

Each fold fits its own models. The boosted model predicts a survival time in days, and days mean the same thing in every fold, so its predictions can be pooled into one list. A Cox score is a risk relative to the other rows in its own fit, so Cox scores from different folds cannot go in one list. The two models are therefore compared on fold means, each fold scored on its own and the 3 scores averaged.

The pooled row answers a different question. It scores all test rows in one list, which is enough data to attach an uncertainty range, and that range is the interval beside it. It is not the number for comparing models, because its rows were scored by 3 different fitted models. On this run it sits below the fold mean.

The boosted model's weakest window is fold 1, starting 1996-01-01, at 0.731. When a cause has been established, the run's notes say so.

The pooled concordance decomposes by `flc_band`. The split answers what kind of skill the score is. A model can be right about which groups end early while telling the rows inside a group apart no better than chance. Such a model uses nothing finer than one number per group, the expressive power of a lookup table. Ranking rows by their group's average gives every row in a group the same score, so comparisons only ever run between groups, and those come out right 65.3% of the time. The boosted model scores each row individually instead, which orders rows inside a group and can also move one past rows in other groups. Inside a group it

is right 68.9% of the time, averaged over the 3 groups big enough to score (at least 50 rows and 10 observed endings), each weighted by its number of comparable pairs.

Figure 2 plots the per-fold concordance from Table 2.

Table 2. Per-fold results. Censoring is the share of each fold's test rows whose ending was not observed. The 5 requested folds merged to 3 where the start dates' granularity gave several the same split date.

FOLD	SPLIT DATE	TRAIN N	TEST N	CENSORED	AFT	COX
1	1996-01-01	1,275	1,890	71%	0.731	0.797
2	1997-01-01	4,765	1,889	75%	0.776	0.785
3	1999-01-01	6,833	944	84%	0.801	0.803

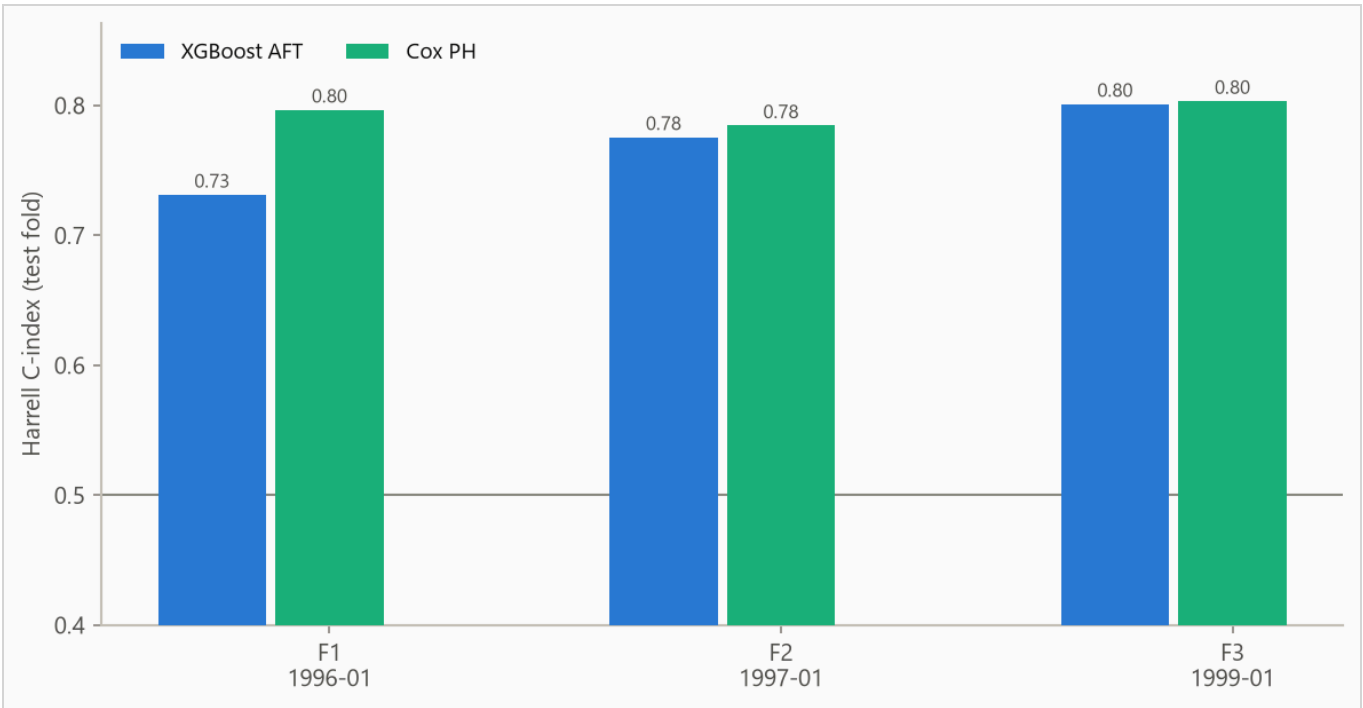


Figure 2. Concordance by fold. The boosted model spans 0.731 to 0.801. Folds differ in censoring mix, so cross-fold comparisons are indicative rather than exact.

5.2 Calibration

Concordance grades only the order of the predictions. This section grades their probabilities, asking whether rows given a 70% chance of surviving a horizon go on to survive it about 70% of the time. The Brier score, the mean squared error of a probability forecast (lower is better), is weighted by the inverse probability of censoring (IPCW) so censored rows do not bias it. The no-skill reference assigns every row one population-

wide probability, the Kaplan-Meier marginal, the fraction of the whole population surviving at each age with censored rows counted while observed.

Table 3. IPCW Brier score by horizon. Lower is better. The weighting works by counting each row still observed at a horizon, weighted up by one over the probability that observation lasted that long, so the rows censoring removed are represented by those it spared.

HORIZON	AFT	COX	NO-SKILL MARGINAL
365 days	0.034	0.030	0.032
730 days	0.059	0.048	0.051
1460 days	0.104	0.081	0.086

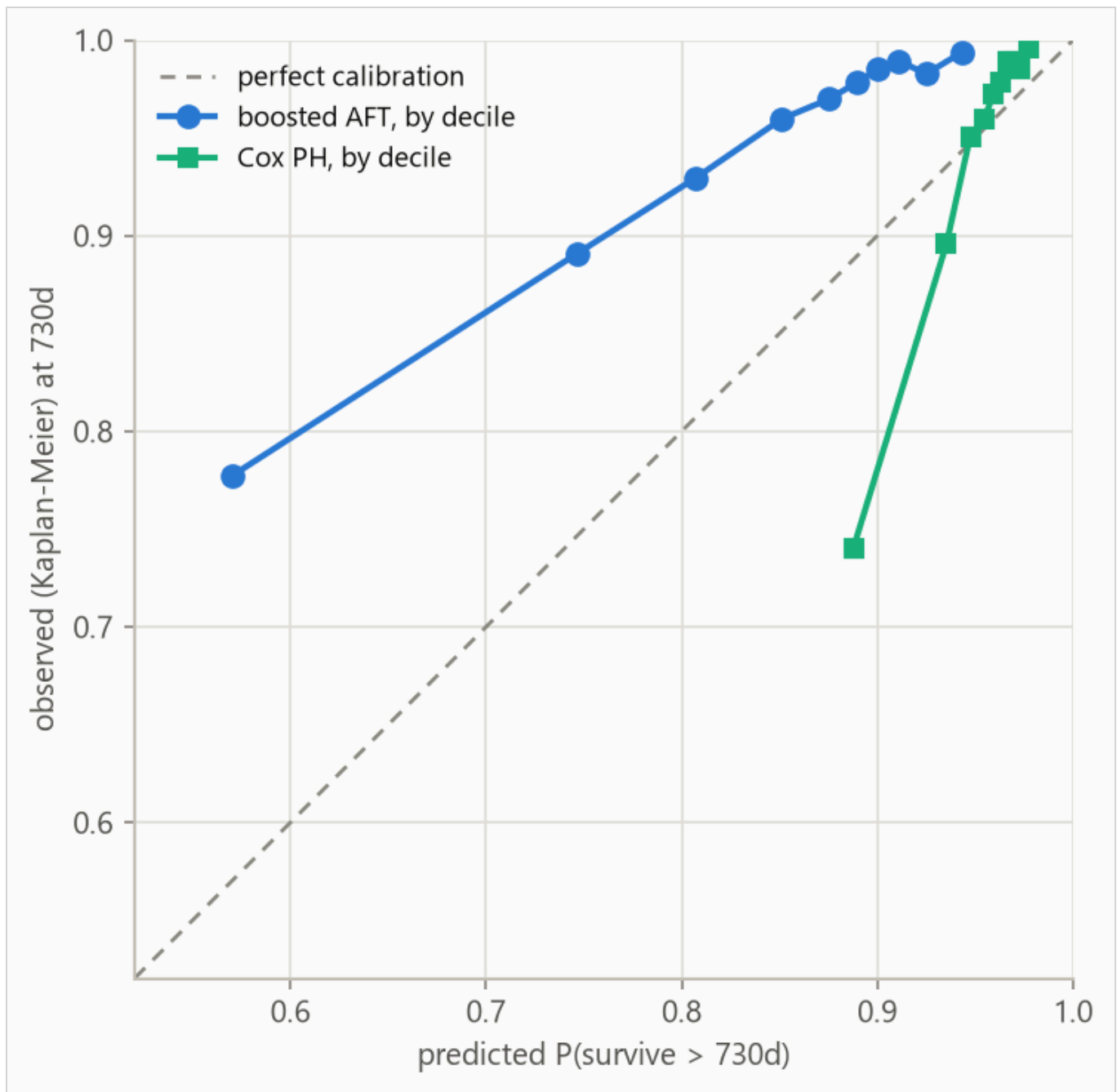


Figure 3. Predicted against observed survival at 730 days for both models, each binned on its own predicted deciles. Observed frequencies are Kaplan-Meier estimates within each bin, so censored rows contribute correctly. The largest deviation is 0.207 in the boosted model's decile 1 and 0.148 in the Cox baseline's decile 1.

Table 4. Decile calibration at 730 days for the boosted AFT model, the values plotted as its series in Figure 3. Deciles are cut on predicted value, so tied predictions make the counts uneven. The observed value in each is a Kaplan-Meier estimate. When a decile's last observed row falls short of the 730-day horizon the estimate carries its last value forward, so in small or heavily censored deciles the observed column carries more uncertainty than its three decimals suggest.

DECILE	N	PREDICTED	OBSERVED (KM)	DEVIATION
1	473	0.570	0.777	+0.207
2	472	0.747	0.891	+0.144
3	472	0.807	0.930	+0.122
4	476	0.851	0.960	+0.108
5	469	0.875	0.970	+0.094
6	472	0.890	0.978	+0.089
7	472	0.901	0.985	+0.085
8	472	0.911	0.989	+0.078
9	472	0.925	0.983	+0.058
10	473	0.943	0.994	+0.051

6. What the models use

6.1 The boosted model

Feature attributions (SHAP values, for SHapley Additive exPlanations) are computed for the boosted model on the log scale of survival time. A +0.3 attribution multiplies predicted survival by about 1.35, and negative values shorten it.

Attributions are descriptive and in-sample, computed on the final boosted model's own training rows. Correlated features split credit by the model's internal choices as much as by the data, so directions are more trustworthy than magnitudes.

Figure 4 ranks features, Table 5 lists the top eight, Figure 5 shows the per-row spread, and Figure 6 traces attribution against value.

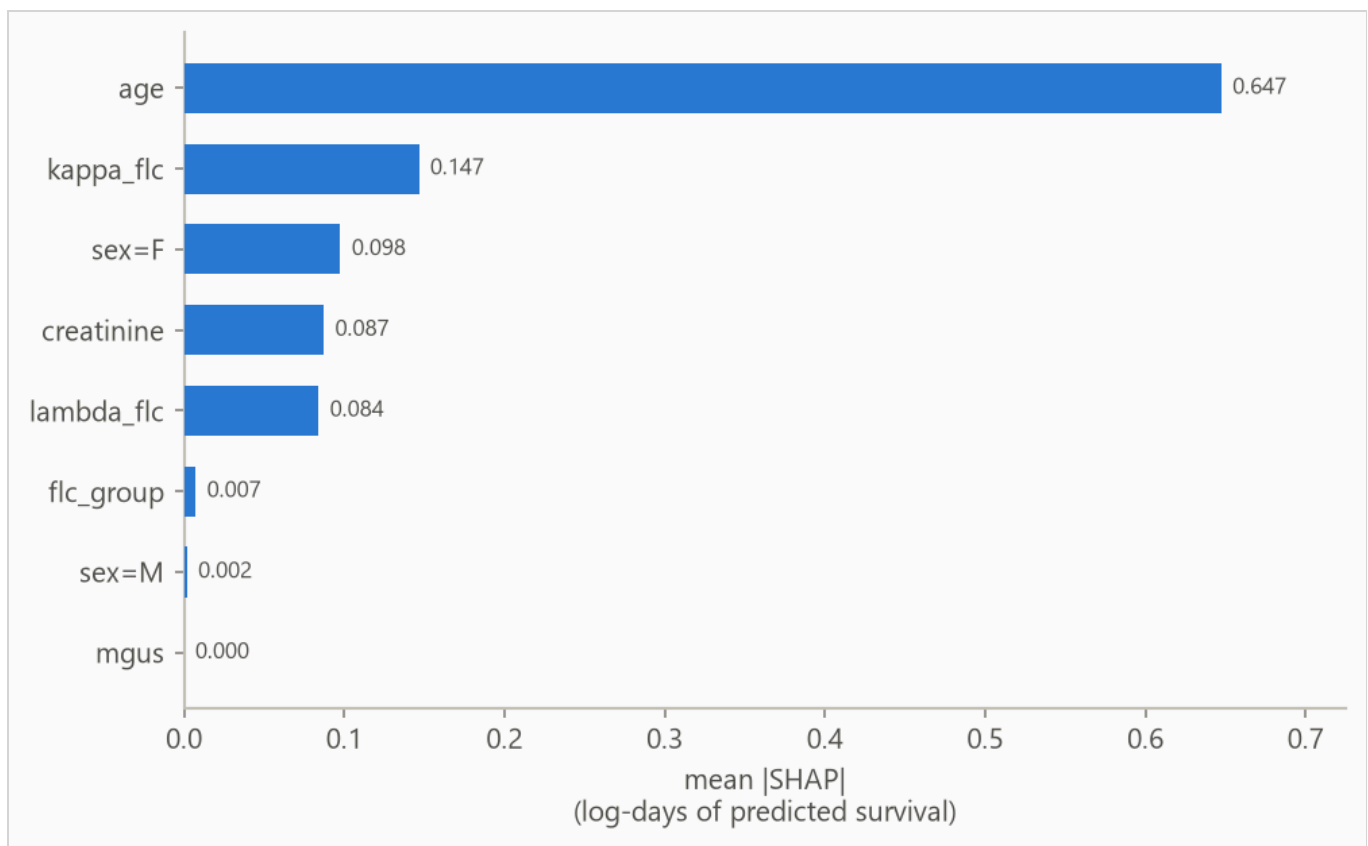


Figure 4. Mean absolute attribution by feature.

Table 5. Top eight features by mean absolute attribution.

FEATURE	MEAN ATTRIBUTION
age	0.647
kappa_flc	0.147
sex=F	0.098
creatinine	0.087
lambda_flc	0.084
flc_group	0.007
sex=M	0.002
mgus	0.000

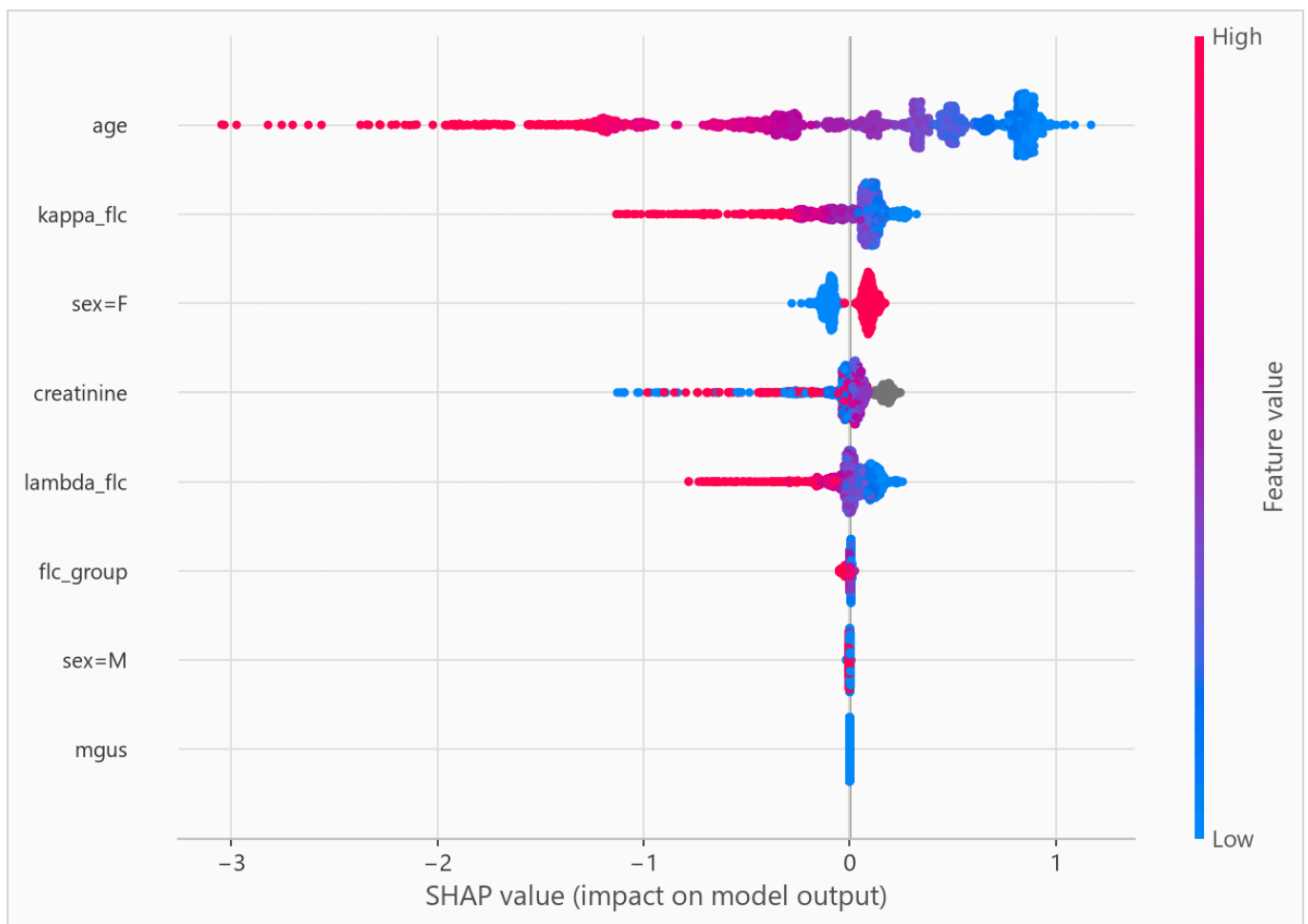


Figure 5. Per-row attributions across the explanation sample. For a yes/no feature, such as a one-hot category flag, the high (red) end simply means the row is in that category.

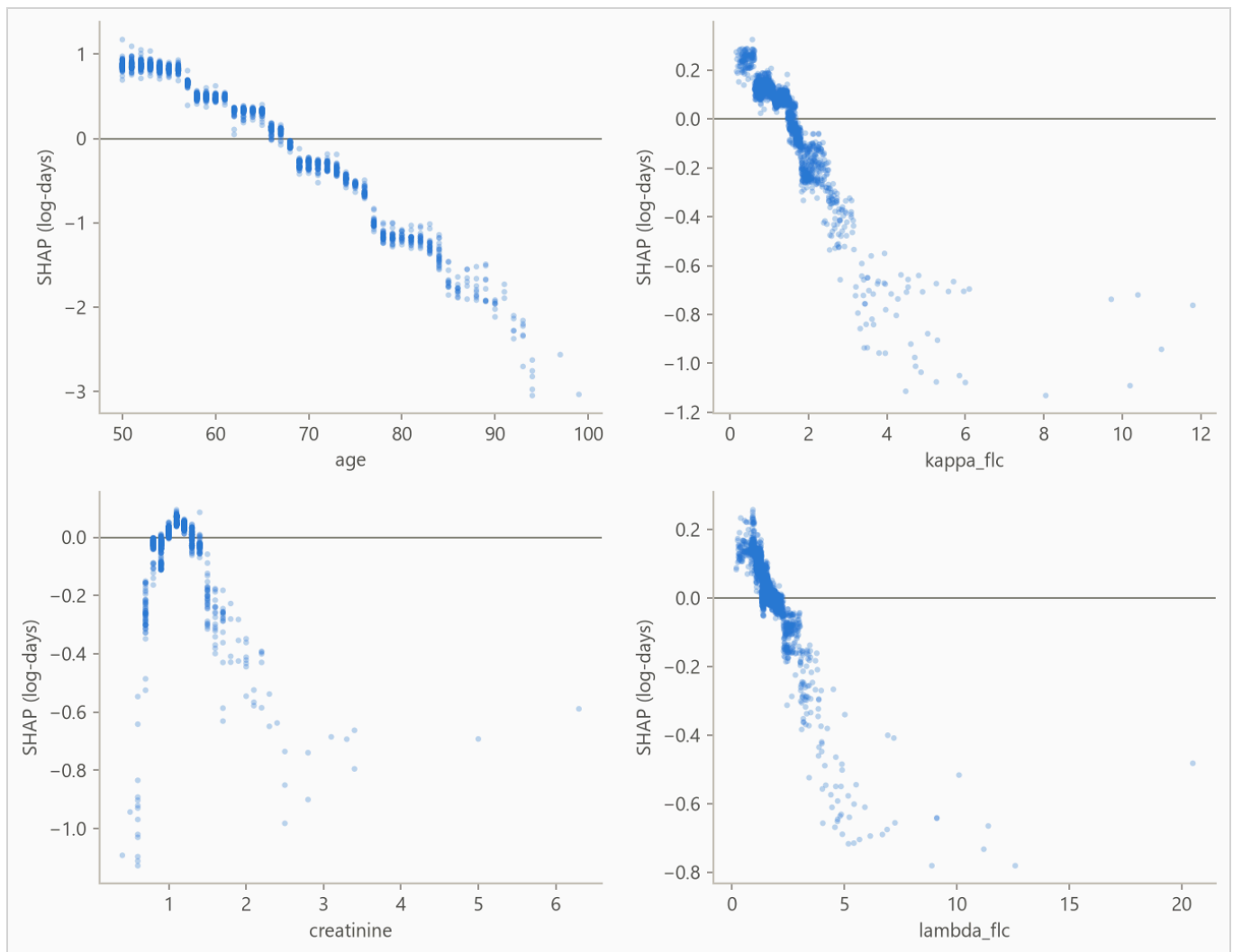


Figure 6. Attribution against feature value for the strongest numeric features. Category flags carry no shape and stay in the ranking above. A run short on numeric features fills the grid with flags instead.

On this data there is no known mechanism to check the ranking against, so read it as "what this model leaned on", not as importance in the world.

6.2 The Cox baseline

The Cox baseline's drivers are its coefficients, reported as hazard ratios. A hazard ratio is the factor by which one unit of a feature multiplies the hazard, the risk of ending at a given age. The factor is the same at every age, so a ratio above 1 shortens survival, the opposite reading from the attributions above. Figure 7 plots the strongest covariates, ranked by $|z|$, the coefficient over its standard error, and Table 6 lists the top eight.

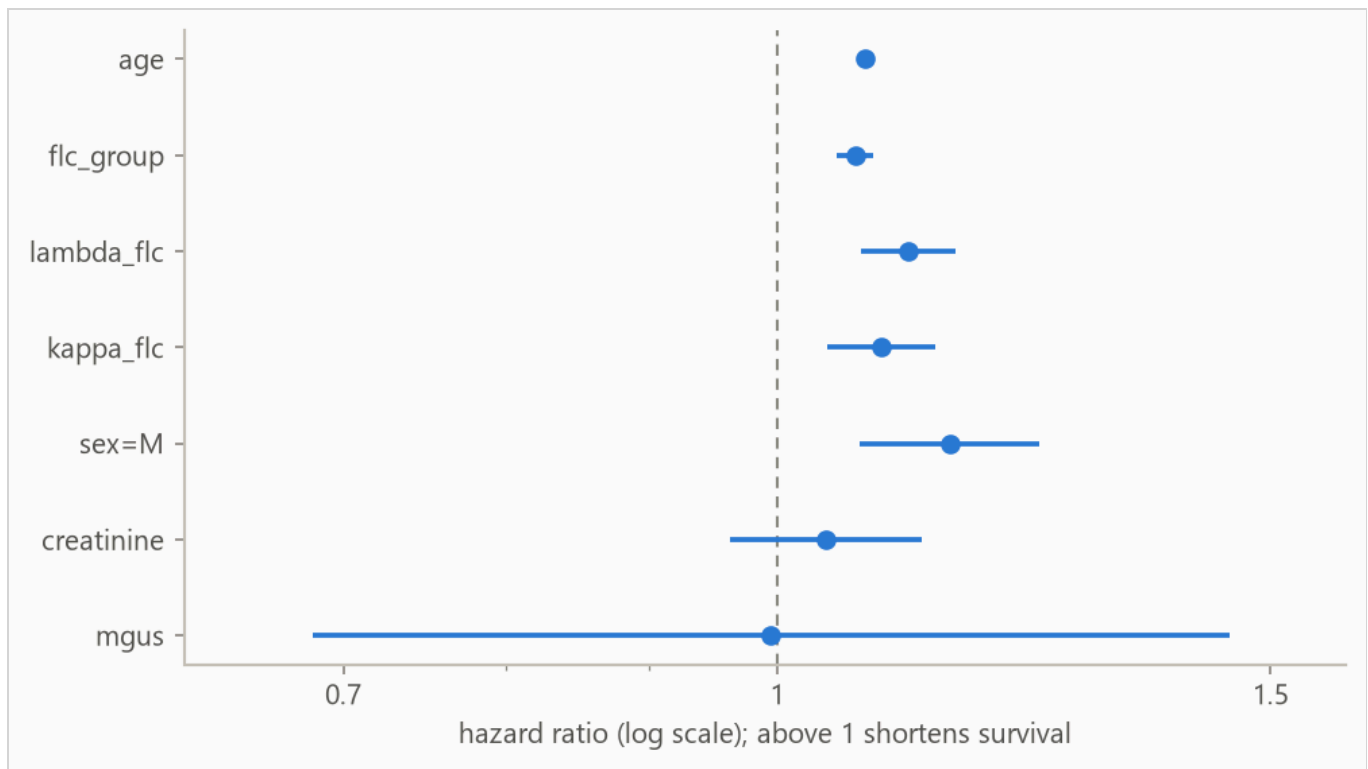


Figure 7. Hazard ratios for the strongest Cox covariates on a log axis, so a doubling and a halving of risk sit the same distance from the dashed no-effect line at 1.0.

Table 6. Top eight Cox covariates by $|z|$. A ratio above 1 raises the hazard and shortens survival. A one-hot flag's ratio compares its category against the column's reference level, the one category dropped from the Cox fit so the remaining flags stay independent. The ridge penalty that stabilizes the fit shrinks coefficients toward zero, so the 95% intervals are approximate.

FEATURE	HAZARD RATIO	95% INTERVAL	Z
age	1.074	1.071 to 1.078	+38.5
flc_group	1.066	1.051 to 1.082	+8.5
lambda_flg	1.114	1.072 to 1.158	+5.5
kappa_flg	1.089	1.042 to 1.139	+3.8
sex=M	1.152	1.070 to 1.241	+3.7
creatinine	1.041	0.962 to 1.126	+1.0
mgus	0.995	0.682 to 1.451	-0.0

7. Interpretation

From the run's notes, written by the analyst.

How this compares to the standard exercise. This dataset is a teaching classic, and the usual exercise evaluates it with a random split. A random subset of people is hidden, the model trains on the rest, and its training pool therefore spans every year of the study. A benchmark paper (BigSurvSGD, arXiv:2003.00116) reports a concordance around 0.794 under that protocol.

A random split suits the original research goal. A researcher asking whether the blood assay predicts mortality beyond age wants to know whether the relationship is real. The whole completed study is fair evidence for that, and the deliverable is the finding itself. Training only on the past suits the deployment goal. A clinic scoring each new patient at intake stands at a moment in time and knows only what has happened so far, and a test is only honest if it rehearses that position. This pipeline runs the second protocol and reaches the same answer. The Cox baseline's fold mean is 0.795, matching the benchmark figure, with the three folds at 0.797, 0.785, and 0.803.

What the review process caught. An earlier version of this run reported a different result, and the review that retired it is worth recording. The source file lists subjects within each sample year in age order. Start dates here are year-only, so a fold boundary that lands inside a year cannot cut on time and cuts between rows instead, and row order here was age order. Two folds trained on identical windows and scored 0.649 and 0.847 on the unshuffled file, purely from who landed in their test slices. The preparation script now shuffles within each year with a fixed seed, folds that snap to the same split date merge into one, and the horizons now sit within the longest training window's observed follow-up. A random split can never meet this failure, because it never cuts along the file at all. A temporal protocol on coarse dates has to, which is why the run documents it.

Why the boosted model still trails. Only 28% of subjects died while the study watched. Under that much censoring the boosted model trails the Cox baseline on ranking, 0.769 against 0.795 on the fold mean, and on probability quality, running slightly behind the no-skill forecast at every horizon while Cox runs slightly ahead. The Brier table in the results section grades this, and lower is better. The boosted model fits a fixed curve shape and guesses lifetimes from mostly unfinished ones. Cox reads its curve off the data by counting who is still alive at each age, which holds up where the data can check it. This is why the pipeline fits both models and lets the data choose, and this run's bundle recommends Cox.

What the attribution shows. Age carries most of the ranking, and the two light-chain measurements add signal on top, which is the original study's own finding. The decomposition by the assay's banding makes the model's contribution precise. A low, middle, and top banding of the ten assay groups alone ranks survival at 0.653, and comparisons restricted to subjects within the same band score 0.689, so the model adds real ranking beyond the assay's own grouping. The Kaplan-Meier figure in the data section shows the banding's separation directly.

8. Limitations

Absolute probabilities are not usable at 365, 730, and 1460 days. The boosted model loses to the no-skill forecast on the censoring-weighted Brier score there, and Table 3 grades each model separately. Ranking and probability quality are separate, so ordering rows is still supported at those horizons.

Censoring may be informative. The evaluation assumes rows stop being observed for reasons unrelated to their risk. Early exits of rows about to end anyway bias survival estimates upward.

From the run's notes, written by the analyst.

Folds merge where the dates cannot separate them. Start dates are years, so several requested folds can snap to the same split date. Those merge into one fold with the combined test block rather than posing as distinct models, which is why the report shows three folds against the five the reproduce command requests.

Year-floored dates leak a little future into training labels. Every start is recorded as January 1 of its sample year, backdating subjects by up to a year. Re-censoring therefore keeps some deaths that truly happened just after a split inside that split's training labels. The bias runs toward optimism, is bounded by the deaths falling within a year of each split, and finer dates would remove it. Its size is not measured here.

Horizons stay inside the longest window's observation. The training windows watch subjects for one to four years, so the report grades probabilities at one, two, and four years only. An earlier version of this run asked these windows for ten-year probabilities and got extrapolations dressed as measurements. Support is still per fold. The one-year row is inside every window's observation, while the four-year row leans on the widest window, and the earlier folds contribute extrapolated predictions there. The Brier rows in the results section grade all of it, and lower is better there.

The boosted model gives every row one curve shape. The predictive scale is a single fitted number, so the model runs a row's clock faster or slower but never changes the curve's shape. Per-row width is future work.

Harrell's concordance is biased under heavy censoring, usually upward. It scores only the pairs censoring leaves comparable, which under heavy censoring over-represent short observed lifetimes. The most censored test window is 84% censored, so read its rows in Table 2 with the most doubt.

9. Reproducing this run

Python 3.11 or later. From the repository root:

```
python scripts/run_fit_evaluate.py --data datasets/flchain.csv.gz --name flchain --id-col subject.  
python scripts/run_build_report.py --run reports/flchain_demo
```

Every number is read from the run's `metrics.json` at build time, and a figure the prose never cites fails the build. `requirements.txt` pins the exact versions the committed numbers came from.

Generated from `reports/flchain_demo/metrics.json` by `scripts/run_build_report.py --run .` 7,871 rows, 4,723 out-of-time test rows.